YENEPOYA
(Deemed to be University)

Yenepoya Institute of Arts Science Commerce and Management

Project-Low Level Design

on

Multi cloud hybrid strategy for smart city

Student Name: Muhammed Abdul Gafoor

Industry mentor:mr.Sumit kumar shukla **Guided by:Deepthi k**

Table of Contents

1.1. Introduction 3
1.2. Scope of the document. 4
1.3. Intended Audience 4
1.4. System overview. 5

2. System Design 6
2.1. Application Design 7
2.2. Process Flow. 8
2.3. Information Flow 9
2.4. Components Design 9
2.5. Key Design Considerations 11
2.6. API Catalogue 12

3. Data Design 12
3.1. Data Model 12
3.2. Data Access Mechanism 13
3.3. Data Retention Policies 13
3.4. Data Migration 14

4. Interfaces 14

5. State and Session Management 15

6. Caching 15

7. Non-Functional Requirements 15
7.1. Security Aspects 16
7.2. Performance Aspects 16

8. References 16

1. Introduction

The **Low-Level Design (LLD)** of the *Multi-Cloud Hybrid Strategy for Smart City* system provides a detailed description of how the system is implemented at a functional and module level. While the High-Level Design focuses on overall architecture, this document explains the internal workings of each component, including how data flows, how operations are executed, and how different modules interact with each other.

This system is designed to efficiently manage citizen complaints while ensuring data reliability through a hybrid cloud strategy. The LLD focuses on how the backend processes requests, manages data, and performs backup and recovery operations automatically.

The document explains the working of critical internal processes such as:

- **Database operations**, including storing, retrieving, and updating complaint data
- **Backup mechanisms**, which ensure periodic saving of system data
- **Restore processes**, which allow system recovery during failures
- **Signal-based triggers**, which automate backup operations without manual intervention

This level of detail helps developers understand exactly how the system behaves internally and supports easier implementation and debugging.

1.1 Scope of the Document

This document covers the internal design and implementation details of the system. It provides a comprehensive view of how each module operates and interacts with other components.

The scope includes:

- Detailed description of all system modules such as user management, request handling, backup, and restore
- Explanation of class-level and function-level design for key operations
- Internal workflows including complaint processing, assignment, and feedback handling
- Backup and recovery logic, including how failover is implemented

The document focuses on system functionality and internal logic rather than user interface or high-level architecture.

1.2 Intended Audience

This document is intended for various stakeholders involved in the project:

- **Developers**
  To understand module structure, logic flow, and function implementation for coding and debugging
- **Testers**
  To design test cases based on workflows and validate system behavior
-

- **System Implementers**
  To deploy and configure the system correctly in a production environment
- **Academic Evaluators**
  To assess the technical depth, completeness, and correctness of the design

---

1.3 System Overview

The system is a **Smart City Complaint Management Platform** designed to streamline communication between citizens and authorities.

Core Components

- **Django Backend**
  Handles business logic, request processing, and workflow management
- **MySQL Database**
  Stores structured data such as users, complaints, and assignments
- **Python Backup Scripts**
  Automate backup and recovery operations

System Capabilities

- **Data Redundancy**
  Data is stored in both local storage and cloud storage to prevent loss
- **Failover Support**
  If the primary system fails, backup data can be used for recovery
- **Automatic Backup Triggering**
  Backup processes are triggered using Django signals and schedulers without manual intervention

---

2. System Design (Expanded)

2.1 Application Design (Expanded)

The system is built using the **Django MVT architecture**, which ensures separation of concerns and improves maintainability.

Model Layer

- Defines database tables using Django models
- Handles data validation and constraints
- Maintains relationships between entities using foreign keys

Example:

- User model stores user details
- Request model stores complaint data

View Layer

- Processes incoming requests from users
- Contains business logic for handling operations
- Interacts with models to store or retrieve data
- Triggers backup processes using signals

Template Layer

- Displays data to users through web pages
- Provides forms for input and dashboards for monitoring
- Uses Django template engine for dynamic content

Additional Scripts

backup.py

- Executes full backup of database and media
- Handles file compression and storage
- Ensures data is saved in both local and cloud

restore.py

- Restores system data from backup
- Supports recovery from both local and cloud storage
- Ensures minimal downtime during failure

2.2 Process Flow

Request Process

1. User submits a complaint through the system
2. Input data is validated for correctness
3. Data is stored in the database
4. Django signal is triggered after saving
5. Backup process starts automatically
6. Admin assigns the complaint to an authority
7. Authority updates status until resolution
8. User provides feedback after completion

Backup Process

1. Backup is triggered by scheduler or signal
2. Database dump is created in SQL format
3. Media files are compressed into ZIP format
4. Backup is stored locally with timestamp
5. Backup is uploaded to cloud storage
6. System verifies successful upload
7. Old backups are deleted based on policy

Restore Process

1. System checks for available local backup
2. If not available, cloud backup is accessed
3. Database is restored using SQL dump

4. Media files are extracted
5. System verifies successful restoration

2.3 Information Flow

The system follows a structured flow of data:

- User sends request to Django View
- View processes data and interacts with database
- After saving data, backup script is triggered
- Backup is stored in local and cloud storage

This ensures that all critical data is safely stored and recoverable.

2.4 Components Design

Authentication Module

- Manages login and registration
- Uses Django authentication system
- Ensures secure access using sessions

User Module

- Stores user information
- Allows users to submit and track complaints

- Maintains user history

Request Module

- Handles complaint submission and tracking
- Stores details such as description and status
- Maintains lifecycle of requests

Assignment Module

- Assigns complaints to authorities
- Tracks progress and updates

Feedback Module

- Collects user feedback after resolution
- Helps improve system performance

Backup Module (Expanded)

Functions:

- `run_full_backup()` → Executes full backup
- `backup_database()` → Creates database dump
- `backup_media()` → Compresses media files

Responsibilities:

- Ensure data consistency during backup
- Handle errors during backup
- Upload data to cloud storage
- Maintain backup history

Restore Module

Functions:

- `run_restore()` → Initiates restore process
- `restore_db()` → Restores database

Responsibilities:

- Retrieve backup from storage
- Restore system data accurately
- Implement failover mechanism

2.5 Key Design Considerations

- **Modularity**
  System is divided into independent modules for easier maintenance
- **Error Handling**
  Uses exception handling to avoid system crashes
- **Failover**
  Backup data is used when primary system fails
- **Efficiency**
  Multithreading is used for faster backup operations

- **Maintainability**
  Clean and structured code improves maintainability

2.6 API Catalogue

Each API function performs a specific operation:

- `login_get()` - Authenticates user
- `user_register()` - Registers new users
- `user_sendreq()` - Submits complaint
- `admin_assignauthority()` - Assigns authority
- `backup_database()` - Creates DB backup
- `backup_media()` - Handles media backup
- `run_restore()` - Restores system

---

3. Data Design

3.1 Data Model (Expanded)

The system uses a relational database with multiple interconnected tables.

Tables

- user_table → Stores user details

- authority → Stores authority information
- request_table → Stores complaints
- assign_authority → Links requests and authorities
- feedback → Stores user feedback

Relationships

- One user can create multiple requests
- Each request is assigned to one authority
- Feedback is linked to completed requests

These relationships are maintained using foreign keys.

3.2 Data Access Mechanism

Django ORM is used for database operations:

- Insert → `.save()`
- Retrieve → `.objects.get()`
- Filter → `.objects.filter()`
- Update → `.update()`

This reduces complexity and ensures secure interaction with the database.

3.3 Data Retention Policies

- Stores last 3 database backups
- Stores last 2 media backups
- Automatically deletes older backups

This helps manage storage efficiently and ensures recent recovery points.

3.4 Data Migration (Expanded)

- Managed using Django migration tools
- Used for updating database schema
- Backup files assist in recovery
- Ensures smooth upgrades without data loss

---

4. Interfaces (Expanded)

Internal Interfaces

- Interaction between Views, Models, and Backup scripts
- Ensures smooth communication between system components

External Interfaces

- Google Drive API for cloud storage
- MySQL database for persistent storage

---

5. State and Session Management (Expanded)

- Uses Django session framework
- Maintains logged-in user state
- Supports role-based access control
- Ensures secure session handling

---

6. Caching (Expanded)

- Currently not implemented
- Can be added to improve performance
- Tools like Redis can be used
- Helps reduce database load

---

7. Non-Functional Requirements

7.1 Security Aspects

- Authentication using Django
- Role-based authorization
- Secure password storage
- OAuth for cloud access

7.2 Performance Aspects

- Multithreading for backup
- Scheduled jobs using APScheduler
- Efficient file handling
- Optimized storage usage

8. References

- Python Documentation
- Django Documentation
- MySQL Documentation
- PyDrive2 Documentation
- Cloud Computing Concepts